



CHAPTER 3

EFFICIENCY OF AN ALGORITHM

Learning Outcomes

At the end of this lecture, you should be able to explain:

- ▶ the goal of algorithm analysis
- ▶ running time Analysis
- ▶ and compare algorithms
- ▶ rate of growth
- ▶ types of analysis

Analysis of Algorithm

- ▶ To go from city “A” to city “B”, there can be many ways of accomplishing this: by flight, by bus, by train and also by bicycle.
- ▶ Depending on the availability and convenience, we choose the one that suits us.
- ▶ Similarly, in computer science, multiple algorithms are available for solving the same problem (for example, a sorting problem has many algorithms, like insertion sort, selection sort, quick sort and many more).
- ▶ Algorithm analysis helps us to determine which algorithm is most efficient in terms of time and space consumed.

Goal of Algorithm Analysis

Compare algorithms (or solutions)

1. Running time
2. Memory requirement
3. Developer effort

Running Time Analysis

- ▶ Process of determining how processing time increases as the size of the problem (input size) increases.
- ▶ Input size is the number of elements in the input (may be of different types)

Common Types of Input

- ▶ Size of array
- ▶ Polynomial degree
- ▶ Number of elements in the matrix
- ▶ Number of bits in the binary representation of the input
- ▶ Vertices and edges in a graph

How to Compare Algorithms

- ▶ Execution Times?
 - ▶ Not a good measure as execution time are specific to a particular computer
- ▶ Number of Statements executed?
 - ▶ Not a good measure, since the number of statements varies with the programming language as well as style of the individual programmer

Algorithm Efficiency

- ▶ You might implement algorithms and run the programs to get execution time.
- ▶ But there are two problems with this approach:
 - ▶ First, many tasks run concurrently on a computer. The execution time of a particular program depends on the system load.
 - ▶ Second, the execution time depends on specific input. Consider, for example, linear search and binary search.
- ▶ If an element to be searched happens to be the first in the list, linear search will find the element quicker than binary search.

How to Compare Algorithms

- ▶ Ideal Solution?
 - ▶ Let us assume that we express the running time of a given algorithm as a function of the input size n (i.e., $f(n)$) and compare these different functions corresponding to running times.
 - ▶ This kind of comparison is independent of machine time, programming style, etc.

Rate of Growth

- ▶ It is very difficult to compare algorithms by measuring their execution time.
- ▶ To overcome these problems, a theoretical approach was developed to analyze algorithms independent of computers and specific input.
- ▶ This approach approximates the effect of a change on the size of the input.
- ▶ In this way, you can see how fast an algorithm's execution time increases as the input size increases, so you can compare two algorithms by examining their growth rates.

Rate of Growth

- ▶ The rate at which the running time increases as a function of input

Rate of Growth (Example)

- ▶ Assume that you go to a shop to buy a car and a bicycle.
- ▶ If your friend sees you there and asks what you are buying, then in general you say buying a car.
- ▶ This is because the cost of the car is high compared to the cost of the bicycle (approximating the cost of the bicycle to the cost of the car).

$$\begin{aligned} \text{Total Cost} &= \text{cost_of_car} + \text{cost_of_bicycle} \\ \text{Total Cost} &\approx \text{cost_of_car} \text{ (approximation)} \end{aligned}$$

Rate of Growth (Example)

- ▶ we can represent the cost of the car and the cost of the bicycle in terms of function, and for a given function ignore the low order terms that are relatively insignificant (for large value of input size, n).

- ▶
$$n^4 + 2n^2 + 100n + 500 \approx n^4$$
function and approximate to n^4 since n^4 is the highest rate of growth.

Rate of Growth

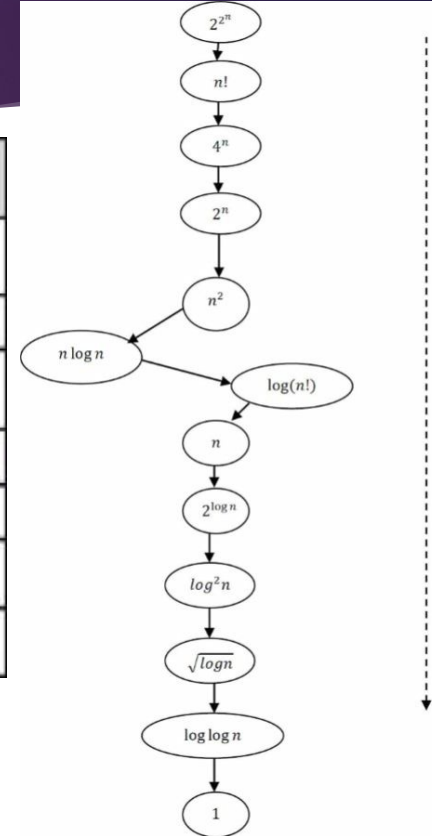
- ▶ Consider linear search.
- ▶ The linear search algorithm compares the key with the elements in the array sequentially until the key is found or the array is exhausted.
- ▶ If the key is not in the array, it requires n comparisons for an array of size n .
- ▶ If the key is in the array, it requires $n/2$ comparisons on average.
- ▶ The algorithm's execution time is proportional to the size of the array.

Rate of Growth

- ▶ If you double the size of the array, you will expect the number of comparisons to double.
- ▶ The algorithm grows at a linear rate.
- ▶ The growth rate has an order of magnitude of n .
- ▶ Computer scientists use the Big O notation to represent the “order of magnitude.”
- ▶ Using this notation, the complexity of the linear search algorithm is $O(n)$, pronounced as “order of n .”
- ▶ We call an algorithm with a time complexity of $O(n)$ a linear algorithm, and it exhibits a linear growth rate.

Commonly Used Rates of Growth

Time Complexity	Name	Example
1	Constant	Adding an element to the front of a linked list
$\log n$	Logarithmic	Finding an element in a sorted array
n	Linear	Finding an element in an unsorted array
$n \log n$	Linear Logarithmic	Sorting n items by 'divide-and-conquer' - Mergesort
n^2	Quadratic	Shortest path between two nodes in a graph
n^3	Cubic	Matrix Multiplication
2^n	Exponential	The Towers of Hanoi problem



Types of Analysis

- ▶ To analyze the given algorithm, we need to know
 - ▶ With which inputs the algorithm takes less time
 - ▶ With which inputs the algorithm takes a long time.
- ▶ An algorithm can be represented in the form of an expression.
- ▶ That means we represent the algorithm with multiple expressions:
 - ▶ one for the case where it takes less time and
 - ▶ another for the case where it takes more time.

Types of Analysis

- ▶ In general, the first case is called the best case and
- ▶ the second case is called the worst case for the algorithm.
- ▶ To analyze an algorithm we need some kind of syntax, and that forms the base for asymptotic analysis/notation (Chapter 5).

Types of Analysis

- ▶ Worst case:
 - ▶ Defines the input for which the algorithm takes a long time (slowest time to complete).
 - ▶ Input is the one for which the algorithm runs the slowest.

Types of Analysis

- ▶ Best case:
 - ▶ Defines the input for which the algorithm takes the least time (fastest time to complete).
 - ▶ Input is the one for which the algorithm runs the fastest.

Types of Analysis

- ▶ Average case:
 - ▶ Provides a prediction about the running time of the algorithm.
 - ▶ Run the algorithm many times, using many different inputs that come from some distribution that generates these inputs, compute the total running time (by adding the individual times), and divide by the number of trials.
 - ▶ Assumes that the input is random.

Types of Analysis

Lower Bound $\leq$ Average Time $\leq$ Upper Bound

Types of Analysis

- ▶ For a given algorithm, we can represent the best, worst and average cases in the form of expressions. As an example, let $f(n)$ be the function which represents the given algorithm.

$$f(n) = n^2 + 500, \text{ for worst case}$$

$$f(n) = n + 100n + 500, \text{ for best case}$$

algorithm takes the average running time (or memory).

Algorithm Analysis

- ▶ For the same input size, an algorithm's execution time may vary, depending on the input.
- ▶ An input that results in the shortest execution time is called the best-case input, and an input that results in the longest execution time is the worst-case input.
- ▶ Best-case analysis and worst-case analysis are to analyze the algorithms for their best-case input and worst-case input.
- ▶ Best-case and worst-case analysis are not representative, but worst-case analysis is very useful.

Algorithm Analysis

- ▶ You can be assured that the algorithm will never be slower than the worst case.
- ▶ An average-case analysis attempts to determine the average amount of time among all possible inputs of the same size.
- ▶ Average-case analysis is ideal, but difficult to perform, because for many problems it is hard to determine the relative probabilities and distributions of various input instances.
- ▶ Worst-case analysis is easier to perform, so the analysis is generally conducted for the worst case.

Linear Search Algorithms

- ▶ Linear search is a very simple search algorithm. In this type of search, a sequential search is made over all items one by one.
- ▶ The linear search algorithm requires n comparisons in the worst case and $n/2$ comparisons in the average case if you are nearly always looking for something known to be in the list.
- ▶ Using the Big O notation, both cases require $O(n)$ time.
- ▶ The multiplicative constant ($1/2$) can be omitted.
- ▶ Algorithm analysis is focused on growth rate.
- ▶ The multiplicative constants have no impact on growth rates. The growth rate for $n/2$ or $100n$ is the same as for n , as illustrated in the next table.
- ▶ Therefore, $O(n) = O(n/2) = O(100n)$.

Linear Search Algorithms

Step 1: Input The algorithm takes an array of numbers as input. Let's call this array **arr**.

Step 2: Initialize a variable **max** to store the maximum number found so far.

Step 3: Iterate through the array.

Step 4 : Check for maximum

Step 5: Repeat until the end

Linear Search Example

$n \backslash f(n)$	n	$n/2$	$100n$	
100	100	50	10000	
200	200	100	20000	
	2	2	2	$f(200) / f(100)$



Maximum Number Search Example

- ▶ Consider the algorithm for finding the maximum number in an array of n elements.
- ▶ To find the maximum number if n is 2, it takes one comparison; if n is 3, two comparisons.
- ▶ In general, it takes $n - 1$ comparisons to find the maximum number in a list of n elements.
- ▶ Algorithm analysis is for large input size.

Maximum Number Search Example

- ▶ If the input size is small, there is no significance in estimating an algorithm's efficiency.
- ▶ As n grows larger, the n part in the expression $n - 1$ dominates the complexity.
- ▶ The Big O notation allows you to ignore the nondominating part (e.g., -1 in the expression $n - 1$) and highlight the important part (e.g., n in the expression $n - 1$).
- ▶ Therefore, the complexity of this algorithm is $O(n)$.

Constant Time

- ▶ The Big O notation estimates the execution time of an algorithm in relation to the input size.
- ▶ If the time is not related to the input size, the algorithm is said to take constant time with the notation $O(1)$.
- ▶ For example, a method that retrieves an element at a given index in an array takes constant time, because the time does not grow as the size of the array increases.

Summary

- ▶ Goal of algorithm analysis
- ▶ Running time Analysis
- ▶ Compare algorithms
- ▶ Rate of growth
- ▶ Types of analysis



END OF CHAPTER 3